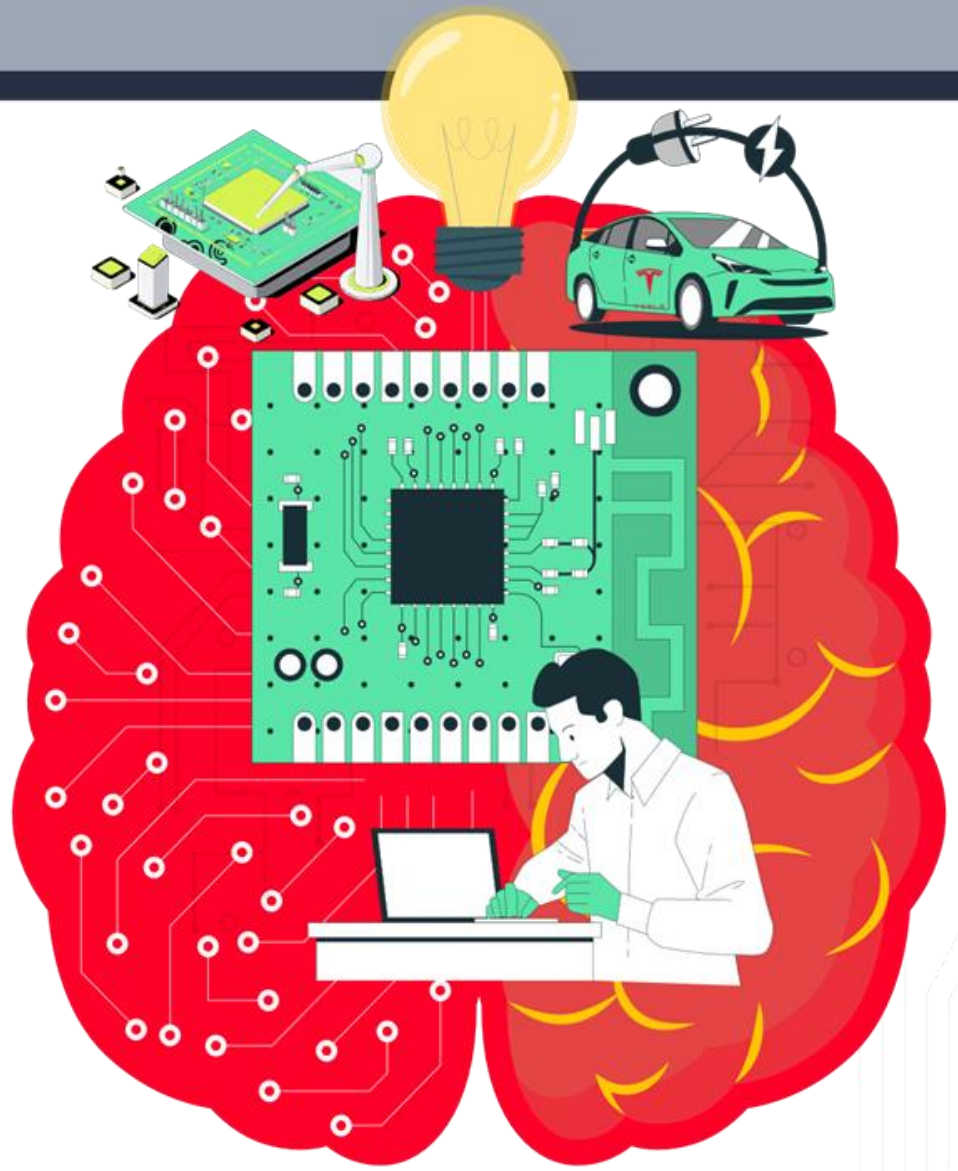


SEMESTER 1 - PROJECT

Customer Support Agent for an Airline Company

SEMESTER 1 PROJECT



SkyAssist: Single-Model Airline Customer Support Agent

Streamlit UI • OpenRouter Free Models • Function Calling • Stateful Memory

7 SESSIONS / 28 HOURS

A capstone application that integrates the Semester 1 agent-building skills.

7-Session Project Roadmap: [for Academic Team]
Each project session leaves a working increment

Session	Focus	Key work	Output
44	Requirements + wireframe	Problem, users, requirements, UI sketch	Approved design
45	Chatbot interface	Streamlit chat + history	Working UI
46	Tools + functions	Mock data + 4–6 functions	Tool layer
47	LLM + function calling	OpenRouter + tool routing	Working agent
48	Stateful assistant	Memory + errors + guardrails	Multi-turn agent
49	Testing + evaluation	10+ scenarios + trace review	Test report
50	Final demo	Packaging + presentation	Final outcome

PROJECT LOGIC Build in increments. Test each layer before adding the next one.

Project:
From classroom building blocks to a working sample support application

- Build one conversational airline support assistant with a browser UI.
- Use a single LLM through OpenRouter and expose controlled Python functions.
- Maintain state so follow-up questions work naturally.
- Validate the agent with realistic test cases and trace evidence.
- Submit code, documentation, test results and a working demo.



Project AIM
Integration is the goal—not just a chatbot

AIM

Build a model-based airline support agent that understands customer requests, decides when a function is needed, returns useful answers and maintains conversation state.

Understand
Interpret natural-language requests and identify missing information.

Act
Call controlled functions for flight, booking, baggage and escalation.

Remember
Carry relevant context across follow-up turns.

SUCCESS A customer can complete a realistic support conversation without knowing which backend function was used.

Project Objectives:
What students must demonstrate

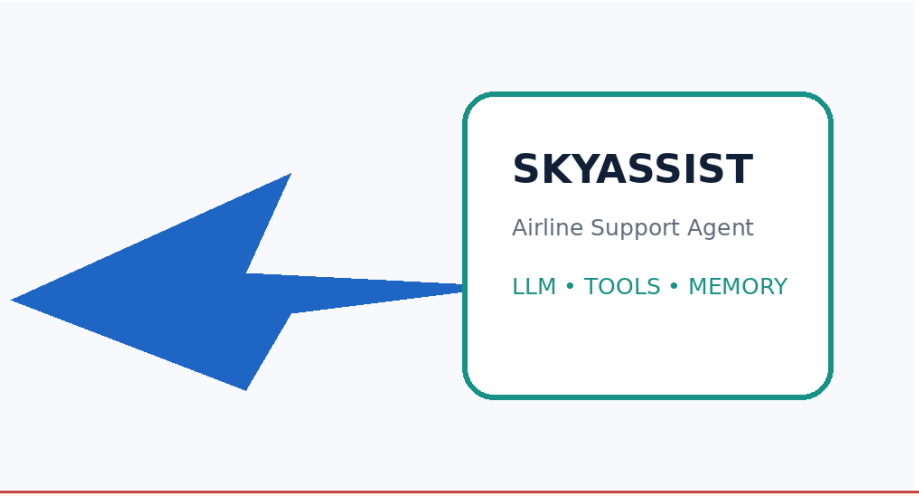
- Translate the airline problem into functional and non-functional requirements.
- Build a Streamlit chat interface with message history.
- Connect an OpenRouter free model and manage the API key safely.
- Define focused tools/functions with structured inputs.
- Implement function calling and return tool results to the model.
- Maintain multi-turn state and handle errors gracefully.
- Create a repeatable test suite and explain the final architecture.

Business Scenario — SkyAir Support
Safe academic simulation using synthetic data

Customer
Needs quick answers without navigating multiple support channels.

Agent
Understands the request, asks for missing details and selects a function when required.

Mock systems
Synthetic flight, booking and policy data exposed through Python functions.



DESIGN BOUNDARY Use synthetic/mock airline data. Never use real passenger records or production airline systems.

Requirements — Functional
What the system must do?

Chat
Accept natural-language questions and show assistant replies.

Flight status
Look up synthetic flight information through a function.

Booking
Retrieve booking information using a mock ID.

Baggage
Answer allowance and calculate an example extra-bag fee.

Flight change
Check simulated alternatives; no real transaction.

Escalation
Create a mock support ticket when needed.

Requirements — Non-Functional
Quality expectations for a classroom-grade prototype

- Usability — first-time users understand what to type.
- Reliability — invalid IDs and unavailable data are handled cleanly.
- Safety — tools only operate on mock data and approved actions.
- Observability — developers can inspect which function was called.
- Maintainability — UI, prompts, tools and data are logically separated.
- Cost awareness — avoid unnecessary model/tool calls.
- Privacy — never commit real PII or API keys.

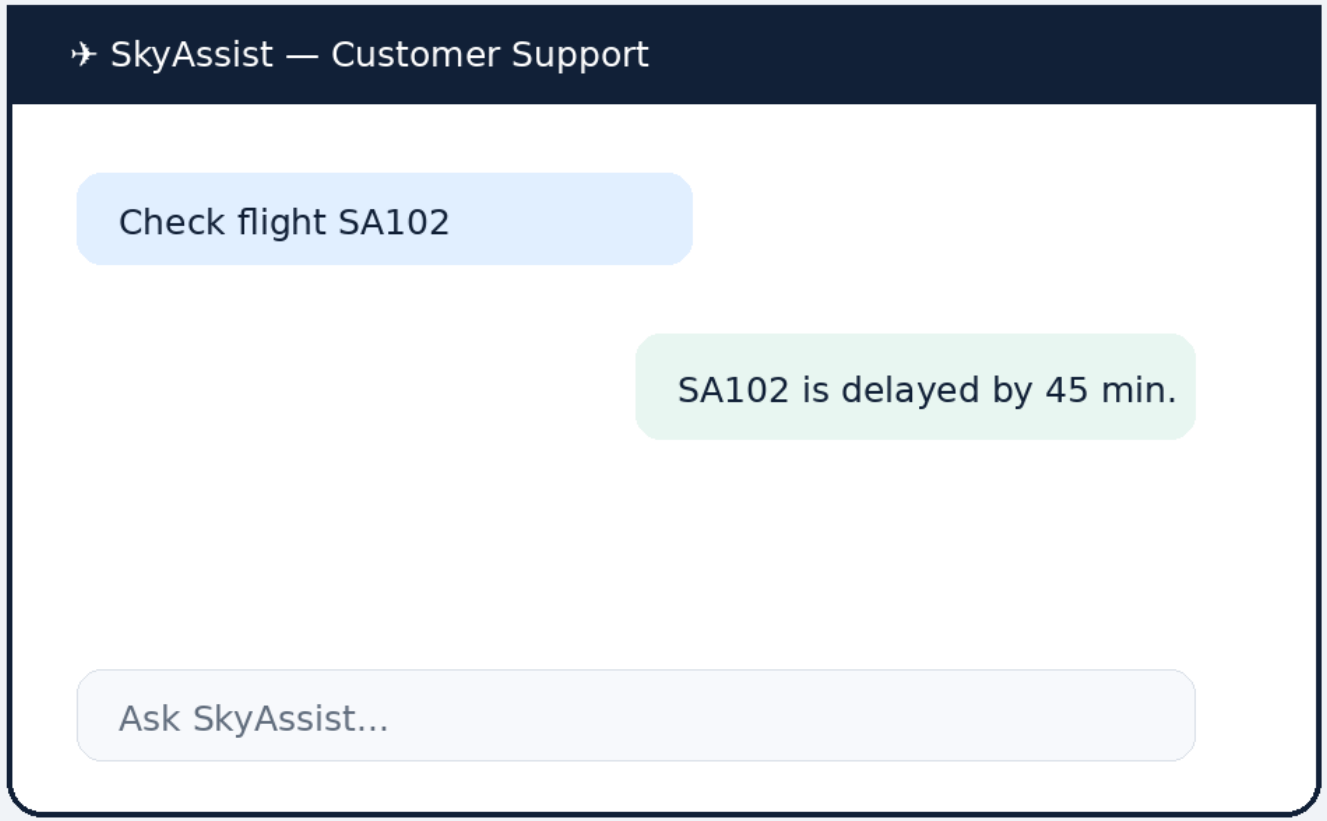
MINIMUM A wrong booking number should never crash the chat application.

Requirement → User Stories
Turn features into testable statements

- As a passenger, I want to check a flight status so I know whether it is delayed.
- As a passenger, I want to look up my booking so I can confirm its status.
- As a passenger, I want follow-up questions to use previous context.
- As support staff, I want unresolved cases escalated with a concise summary.
- As an evaluator, I want evidence of tool calls and test results.

RULE Every important requirement should become at least one test case later.

Wireframe — Streamlit Chat UI
Keep the first version intentionally simple



Header
SkyAssist + one-line description

Chat history
Render user and assistant turns in order.

Input
st.chat_input for the next customer message.

Sidebar
Optional: booking ID, reset, model info.

Agent Loop — Example
Customer: “What happened to flight SA102?”

User
Asks about SA102

LLM
Identifies need

Tool
get_flight_status

Result
Delayed 45 min

LLM
Explains result

TRACE TO SHOW Capture the tool name, arguments, tool result and final response during development.

Test Cases — Core Suite
Minimum 10 scenarios

ID	Type	Example	Expected
TC01	FAQ	Economy baggage allowance?	Correct policy answer
TC02	Flight	Check SA102	Flight tool called
TC03	Booking	Find SKY1001	Booking tool called
TC04	Follow-up	Is that flight delayed?	Uses prior context
TC05	Baggage	Cost for 1 extra bag?	Fee tool called
TC06	Change	Can I change my flight?	Options / clarification
TC07	Invalid booking	Find ABC999	Graceful not-found
TC08	Invalid flight	Check XX000	Graceful not-found
TC09	Unsupported	Refund my card now	Explain / escalate
TC10	Reset	Start new conversation	Old state not reused

PASS Final response and tool behavior must both match the intended design.

Test Cases — What to Observe
Do not grade only the final sentence

Tool selection
Correct function—or unnecessary tool use?

Arguments
Valid and complete inputs?

Result handling
Faithful use of returned data?

Memory
Relevant context without cross-conversation confusion?

Safety
No claims about actions that did not occur?

UX
Clear, concise and useful answer?

EVIDENCE Save screenshots/logs for: successful tool use, a memory follow-up, and a graceful failure.

Demo Scenarios:
Use a short story instead of a random prompt

01 — Flight disruption
“My flight SA102 is delayed. What can I do?”

02 — Booking context
“My booking is SKY1001.” → “Is it delayed?”

03 — Baggage
“How many bags?” → “What if I add one more?”

04 — Escalation
“I need a refund to my card immediately.”

PRESENTATION TIP Narrate what the agent needed, which function it selected, and how the tool result changed the final answer.

Final Project Outcome
What every team should submit?

Working application
Streamlit chat UI connected to the agent.

Agent + tools
Single-model agent with 4–6 tested functions.

State
Conversation state and reset behavior.

Test report
10+ scenarios with pass/fail evidence.

Documentation
Architecture, setup, tool catalog and limitations.

Demo
5–7 minute walkthrough of the finished system.

FINAL OUTCOME A working, explainable prototype—not a production airline system.

Assessment Rubric — 100 Marks [for Academic Team]
Reward engineering quality, not only visual polish

Area	What is assessed	Marks
Problem + architecture	Scope, requirements, design	10
Streamlit UI	Usable conversational interface	15
LLM integration	Correct OpenRouter/model integration	15
Function calling	4–6 tools, routing + inputs	20
State + memory	Multi-turn behavior + reset	10
Error handling	Controlled failures + escalation	10
Testing	10+ scenarios + evidence	10
Code + documentation	Structure + README + reproducibility	5
Demo	Clear explanation + working walkthrough	5

TOTAL 100 marks